



German University in Cairo
Faculty of Engineering and Materials Science

Advanced Mechatronics Engineering MCTR903

Prof Ayman El-Badawy

Lab 3 : System Identification and Parameter Estimation.

Supplementary Material

Contents

1	Introduction to Optimization	2
2	Casting Parameter Estimation as an Optimization Problem	2
3	Handling Cases Without Derivative Data	2
4	Example: Duffing Oscillator Parameter Estimation	3
5	Handling Cases With Noisy Data	4
6	Handling Cases With Output Data	4
7	Example: Duffing Oscillator Parameter Estimation	5

1 Introduction to Optimization

In the field of system identification, **optimization** is essential for estimating unknown parameters in a model. Optimization is the process of finding the best values for certain variables, known as **optimization parameters**, by minimizing or maximizing a specific objective function.

In our case, the optimization parameter is the unknown system parameter, denoted as θ , that we want to estimate. The system's behavior is described by a model of the form:

$$\dot{x}(t) = f(x(t), u(t), \theta)$$

where:

- $x(t)$ represents the system states (e.g., position, velocity),
- $u(t)$ represents the inputs (e.g., forces, voltages),
- θ represents the unknown parameters governing the system dynamics.

The optimization problem can be written as:

$$\min_{\theta} \sum_{t=1}^N \|\dot{x}(t) - f(x(t), u(t), \theta)\|^2$$

The goal is to adjust θ to minimize the difference between the observed data $\dot{x}(t)$ and the model's predictions $f(x(t), u(t), \theta)$.

By solving this optimization problem, we find the value of θ that best fits the observed system behavior, making θ the key **optimization parameter** in parameter estimation.

2 Casting Parameter Estimation as an Optimization Problem

For a continuous system described by:

$$\dot{x}(t) = f(x(t), u(t), \theta)$$

where $x(t)$ represents the system states, $u(t)$ the input, and θ are the unknown parameters, the goal is to estimate θ from observed data.

This can be framed as an optimization problem:

$$\min_{\theta} \sum_{k=1}^N \|\dot{x}(t_k) - f(x(t_k), u(t_k), \theta)\|^2 \quad (1)$$

where $\dot{x}(t)$ are the state derivatives. This optimization is solvable using least squares if the derivatives $\dot{x}(t)$ are known or can be estimated.

3 Handling Cases Without Derivative Data

When the data does not include state derivatives $\dot{x}(t)$, we can discretize the system to remove the derivative $\dot{x}(t)$. Common discretization methods include:

- **Euler Forward:**

$$x_{k+1} = x_k + \Delta t \cdot f(x_k, u_k, \theta) = F_k(x_k, u_k, \theta)$$

- **Euler Backward:**

$$x_{k+1} = x_k + \Delta t \cdot f(x_{k+1}, u_{k+1}, \theta) = F_k(x_k, u_k, \theta)$$

- **Runge-Kutta Methods:** For example, **RK4**:

$$x_{k+1} = x_k + \frac{\Delta t}{6}(k_1 + 2k_2 + 2k_3 + k_4) = F_k(x_k, u_k, \theta)$$

Once the system is discretized, the problem becomes:

$$\min_{\theta} \sum_{k=0}^{N-1} \|x_{k+1} - \hat{x}_{k+1}\|^2 \quad (2)$$

$$\text{subject to: } \hat{x}_{k+1} = F_k(x_k, u_k, \theta) \quad (3)$$

which can be a non-linear least square problem and can be solved using any non-linear optimization software.

4 Example: Duffing Oscillator Parameter Estimation

For the Duffing oscillator:

$$\ddot{x}(t) + \delta\dot{x}(t) + \alpha x(t) + \beta x(t)^3 = \gamma \cos(\omega t)$$

you can either:

- Use numerical differentiation to estimate $\dot{x}(t)$ and $\ddot{x}(t)$,
- Or discretize the system using Euler or Runge-Kutta methods.

Listing 1: MATLAB code for Duffing oscillator parameter estimation

```

1 % MATLAB code for Duffing oscillator parameter estimation
2 % Reading the data from a CSV file
3 data = readmatrix('duffing_data.csv');
4 h=0.02;
5 t=0:h:20; % Time from 0 to 20 seconds with sampling 0.02
6 x_measured = data; % Second and third column are the state variables x, x_dot
7
8 % Define initial guesses for the parameters [delta, alpha, beta, gamma, omega]
9 initial_params = [0, 0, 0, 0, 1.3];
10
11 % Define the objective function to minimize the difference between
12 % measured data and simulated data based on the Duffing oscillator
13 objective_function = @(params) norm(F(x_measured, params) - x_measured, 'fro');
14
15 % Use fmincon to estimate the parameters
16 options = optimoptions('fmincon','Display','iter','TolFun',1e-7,'TolX',1e-7);
17 estimated_params = fmincon(objective_function, initial_params, [], [], [], [], [], [], [],
18     options);
19
20 % Display estimated parameters
21 disp('Estimated Parameters:');
22 disp(['delta = ', num2str(estimated_params(1))]);
23 disp(['alpha = ', num2str(estimated_params(2))]);
24 disp(['beta = ', num2str(estimated_params(3))]);
25 disp(['gamma = ', num2str(estimated_params(4))]);
26 disp(['omega = ', num2str(estimated_params(5))]);
27
28 % Plot measured vs simulated data with the estimated parameters
29 x_estimated=RK4([0 20], [0;0],estimated_params);
30 figure;
31 plot(t, x_measured, 'b', 'LineWidth', 1.5); hold on;
32 plot(t, x_estimated, 'r--', 'LineWidth', 1.5);
33 legend('Measured', 'Simulated');
34 xlabel('Time');
35 ylabel('Displacement');
36 title('Duffing Oscillator Parameter Estimation');
37 grid on;
38
39 %% one step prediction function
40 function x=F(x_measured,params)
41     h=0.02;
42     t_sim = 0:h:20;
43     x = zeros(size(x_measured));

```

```

43 x(:,1)= x_measured(:,1) ;
44 for i = 1:(length(t_sim)-1)
45     temp=RK4([t_sim(i) t_sim(i+1)],x_measured(:,i),params);
46     x(:,i+1) =temp(:,2);
47 end
48 end
49
50 %% Function to simulate Duffing oscillator
51 function x=RK4(t,x0,params)
52     delta = params(1);
53     alpha = params(2);
54     beta = params(3);
55     gamma = params(4);
56     omega = params(5);
57     h=0.02;
58     t_sim = t(1):h:t(2);
59     x = zeros(length(x0),length(t_sim));
60     x(:,1)= x0;
61     % Define the Duffing oscillator ODE
62     duffing_ode=@(t, x) [x(2); -delta*x(2) - alpha*x(1) - beta*x(1)^3 + gamma*cos(omega*t)
63 ];
64 for i = 1:(length(t_sim)-1)
65     k_1 = duffing_ode(t_sim(i),x(:,i));
66     k_2 = duffing_ode(t_sim(i)+0.5*h,x(:,i)+0.5*h*k_1);
67     k_3 = duffing_ode(t_sim(i)+0.5*h,x(:,i)+0.5*h*k_2);
68     k_4 = duffing_ode(t_sim(i)+h,x(:,i)+k_3*h);
69     x(:,i+1) = x(:,i) + (1/6)*(k_1+2*k_2+2*k_3+k_4)*h;
70 end

```

5 Handling Cases With Noisy Data

In practice, the data can be noisy, so we cannot directly use x_k in our prediction of the next state $\hat{x}_{k+1}$. Instead, we need to estimate both the state $\hat{x}_k$ and the parameters θ . The optimization problem becomes:

$$\min_{\theta, \hat{x}_k} \sum_{k=0}^N \|x_k - \hat{x}_k\|^2$$

subject to: $\hat{x}_{k+1} = F_k(\hat{x}_k, u_k, \theta)$

This formulation ensures that we incorporate the estimation of the state alongside the parameters, providing robustness against noisy data. The model attempts to minimize the error between the measured and estimated states while still respecting the system dynamics. You can find the code file on the CMS.

6 Handling Cases With Output Data

In many practical situations, we may not have access to the full state data, but only a subset of the state which is observed as output. In such cases, we modify the optimization problem to handle output data:

$$\min_{\theta, \hat{x}_k} \sum_{k=0}^N \|y_k - \hat{y}_k\|^2$$

subject to: $\hat{x}_{k+1} = F_k(\hat{x}_k, u_k, \theta)$
 $\hat{y}_k = g_k(\hat{x}_k, u_k)$

Here, the objective is to minimize the error between the measured output y_k and the predicted output $\hat{y}_k$. This formulation incorporates both the state and output dynamics, making it suitable for systems where only partial state information is available.

7 Example: Duffing Oscillator Parameter Estimation

For the Duffing oscillator:

$$\ddot{x}(t) + \delta \dot{x}(t) + \alpha x(t) + \beta x(t)^3 = \gamma \cos(\omega t)$$

$$y = x$$

Listing 2: MATLAB code for Duffing oscillator parameter estimation

```

1  % MATLAB code for Duffing oscillator parameter estimation
2  % Reading the data from a CSV file
3  data = readmatrix('duffing_data.csv');
4  h=0.02;
5  t=0:h:20; % Time from 0 to 20 seconds with sampling 0.02
6  x_measured = data; % Second and third column are the state variables x, x_dot
7
8  % Define initial guesses for the parameters [delta, alpha, beta, gamma, omega; x0; v0]
9  initial_params = [0, 0.5, 0.2, 0.1, 1.4, 1.2, -0.1];
10
11 % Define the objective function to minimize the difference between
12 % measured data and simulated data based on the Duffing oscillator
13 objective_function = @(params) simulate_duffing([0 20], params)*[0;1;0] - x_measured
14 % [1;0];
15
16 % Use lsqnonlin to estimate the parameters
17 options = optimoptions('lsqnonlin','Display','iter','TolFun',1e-7,'TolX',1e-7);
18 estimated_params = lsqnonlin(objective_function, initial_params, [], [], options);
19
20 % Display estimated parameters
21 disp('Estimated Parameters:');
22 disp(['delta = ', num2str(estimated_params(1))]);
23 disp(['alpha = ', num2str(estimated_params(2))]);
24 disp(['beta = ', num2str(estimated_params(3))]);
25 disp(['gamma = ', num2str(estimated_params(4))]);
26 disp(['omega = ', num2str(estimated_params(5))]);
27 disp(['x0 = ', num2str(estimated_params(6))]);
28 disp(['v0 = ', num2str(estimated_params(7))]);
29
30 % Plot measured vs simulated data with the estimated parameters
31 ret=simulate_duffing([0 20], estimated_params);
32 time_estimated=ret(:,1);
33 x_estimated=ret(:,[2 3]);
34 figure;
35 plot(time_m, x_measured, 'b', 'LineWidth', 1.5); hold on;
36 plot(time_estimated, x_estimated, 'r--', 'LineWidth', 1.5);
37 legend('Measured', 'Simulated');
38 xlabel('Time');
39 ylabel('Displacement');
40 title('Duffing Oscillator Parameter Estimation');
41 grid on;
42
43 %% Function to simulate Duffing oscillator
44 function ret = simulate_duffing(t, params)
45 % Unpack the parameters
46 delta = params(1);
47 alpha = params(2);
48 beta = params(3);
49 gamma = params(4);
50 omega = params(5);
51 % Initial conditions [x0, v0]
52 x0 = [params(6); params(7)];
53
54 % Define the Duffing oscillator ODE
55 duffing_ode = @(t, x) [x(2); -delta*x(2) - alpha*x(1) - beta*x(1)^3 + gamma*cos(omega*t)];
56
57 % Solve the ODE
58 [time, x_full] = RK4(duffing_ode, t, x0);
59
60 % Return only the displacement (x position) part

```

```
60 x = x_full;
61 ret=[time',x'];
62 end
63
64 function [t_sim,x]=RK4(fun,t,x0)
65 h=0.02;
66 t_sim = t(1):h:t(2);
67 x = zeros(length(x0),length(t_sim));
68 x(:,1)= x0 ;
69 for i = 1:(length(t_sim)-1)
70 k_1 = fun(t_sim(i),x(:,i));
71 k_2 = fun(t_sim(i)+0.5*h,x(:,i)+0.5*h*k_1);
72 k_3 = fun((t_sim(i)+0.5*h),(x(:,i)+0.5*h*k_2));
73 k_4 = fun((t_sim(i)+h),(x(:,i)+k_3*h));
74 x(:,i+1) = x(:,i) + (1/6)*(k_1+2*k_2+2*k_3+k_4)*h;
75 end
76 end
```

Note: In general, the parameter estimation problem is non-linear, and it is important to have a good initial guess of the parameters to avoid getting stuck into a local minimum.